

Training Course

Amazon Web Service



Course Schedule

Day	Presentations	Lab
Day 1	System Operations on AWS	
Day 2	Computing on AWS	X
Day 3	Networking on AWS	X
Day 4	Storage and Archiving in the Cloud	X
Day 5	Monitoring in the Cloud	X
Day 6	Managing Resource Consumption in the Cloud	X

Module 4: Storage S3 in AWS



- **Goal:** Understanding S3 Storage and Data
Lab: Create and configuring S3 Storage

Section introduction

Section introduction

- Amazon S3 is one of the main building blocks of AWS
- It's advertised as “infinitely scaling” storage
- It's widely popular and deserves its own section
- Many websites use Amazon S3 as a backbone
- Many AWS services uses Amazon S3 as an integration as well
- We'll have a step-by-step approach to S3

Amazon S3 Overview - Bucket

- Amazon S3 allows people to store object (files) in “buckets” (directories)
- Buckets must have a globally unique name
- Buckets are defined at the region level
- Naming convention
 - No uppercase
 - No underscore
 - 3-63 characters long
 - Not an IP
 - Must start with lowercase letter or number



Amazon S3 Overview - Object

- Objects (files) have a Key
- The key is the FULL path:
 - S3://my-bucket/my_file.txt
 - S3://my-bucket/my_folder/another_folder/my_file.txt
- The key is composed of prefix + object name
 - S3://my-bucket/my_folder/another_folder/my_file.txt
- There's no concept of “directories” within buckets
- Just keys with very long names that contains slashes (“/”)



Amazon S3 Overview – Object (continued)



- Objects values are the content of the body
 - Max Object Size is 5TB (5000GB)
 - If uploading more than 5GB, must use “multi-part upload”
- Metadata (list of text key/value pairs – system or user metadata)
- Tags (Unicode key/value pair – up to 10) – useful for security/lifecycle
- Version ID (if versioning is enabled)

Amazon S3 Overview – Versioning



- You can version your files in Amazon S3
- It is enabled at the bucket level
- Same key overwrite will increment the “version”: 1,2,3 ...
- It is best practice to version your buckets
 - Protect against unintended deletes (ability to restore a version)
 - Easy roll back to previous version
- Notes:
 - Any file that is not versioned prior to enabling versioning will have version “null”
 - Suspending versioning does not delete previous versions

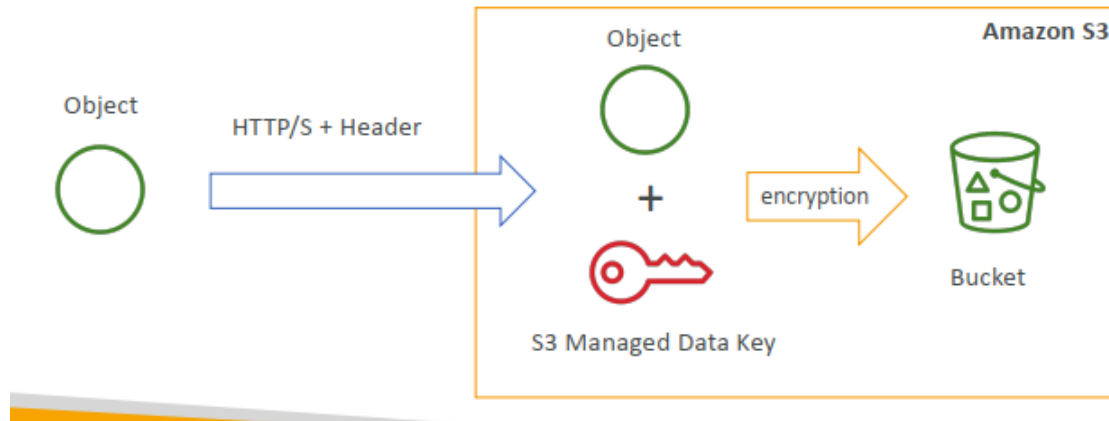
S3 Encryption

S3 Encryption for Object

- There are 4 methods of encrypting object in S3
 - SSE-S3: encrypts S3 objects using keys handled & managed by AWS
 - SSE-KMS: leverage AWS Key Management Service to manage encryption keys
 - SSE-C: when you want to manage your own encryption keys
 - Client Side Encryption
- It's important to understand which ones are adapted to which situation for the exam

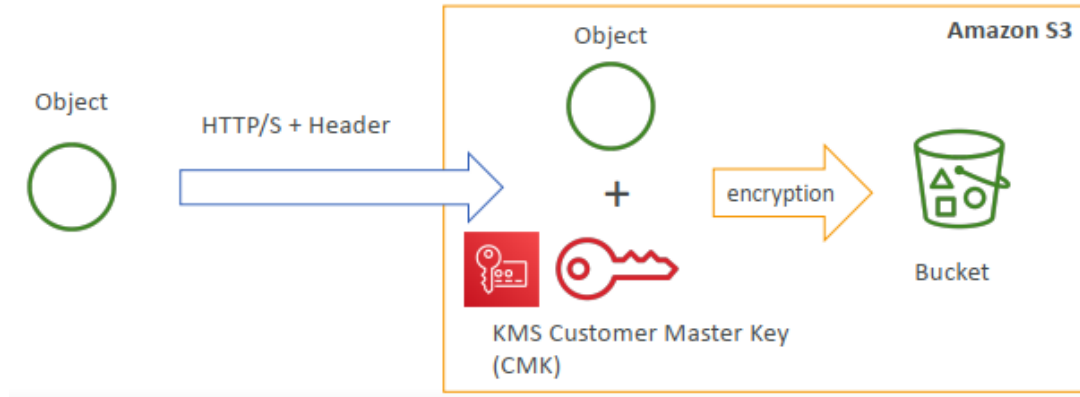
SSE-S3

- SSE-S3: encryption using keys handled & managed by Amazon S3
- Object is encrypted server side
- AES-256 encryption type
- Must set header: “x-amz-server-side-encryption”:”AES256”



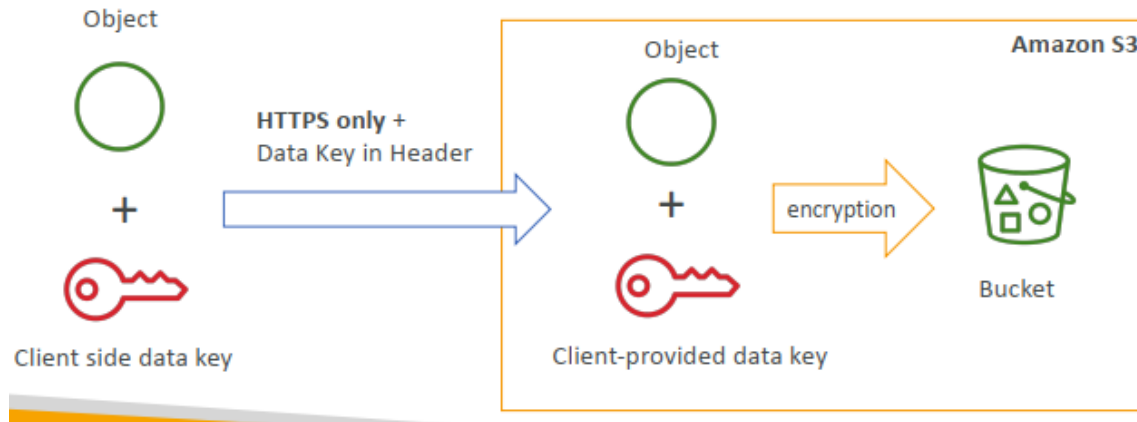
SSE-KMS

- SSE-KMS: encryption using keys handled & managed by KMS
- KMS Advantages: user control + audit trail
- Object is encrypted server side
- Must set header: “x-amz-server-side-encryption”:”aws:kms”



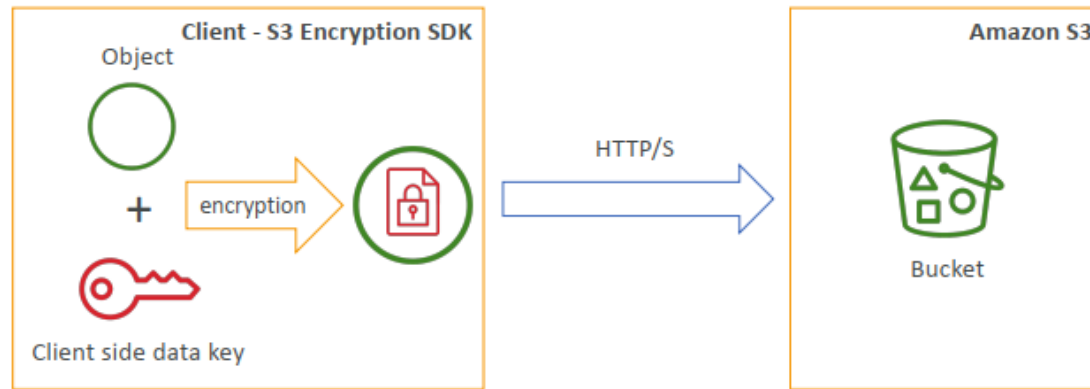
SSE-C

- SSE-C: server-side encryption using data keys fully managed by the customer outside of AWS
- Amazon S3 does not store the encryption key you provide
- HTTPS must be used
- Encryption key must be provided in HTTP headers, for every HTTP request made



Client Side Encryption

- Client library such as the Amazon S3 Encryption Client
- Clients must encrypt data themselves before sending to S3
- Clients must decrypt data themselves when retrieving from S3
- Customer fully manages the keys and encryption cycle



Encryption in transit (SSL/TLS)



- Amazon S3 exposes
 - HTTP endpoint: non encrypted
 - HTTPS endpoint: encryption in flight
- You're free to use the endpoint you want, but HTTPS is recommended
- Most clients would use the HTTPS endpoint by default
- HTTPS is mandatory for SSE-C
- Encryption in flight is also called SSL/TLS

S3 Security



- User based
 - IAM policies – which API calls should be allowed for a specific user from IAM console
- Resource Based
 - Bucket Policies – bucket wide rules from the S3 console – allows across account
 - Object Access Control List (ACL) – finger grain
 - Bucket Access Control List (ACL) – less common
- Note: an IAM principal can access an S3 object if
 - The user IAM permission allow it OR the resource policy ALLOW it
 - AND there's no explicit DENY

S3 Bucket Policies

- JSON based policies
 - Resources: bucket and objects
 - Action: Set of API to Allow or Deny
 - Effect: Allow / Deny
 - Principal: The account or user to apply the policy
- Use S3 bucket for policy to
 - Grant public access to the bucket
 - Force object to be encrypted at upload
 - Grant access to another account (Cross Account)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicRead",
      "Effect": "Allow",
      "Principal": "*",
      "Action": [
        "s3:GetObject"
      ],
      "Resource": [
        "arn:aws:s3:::examplebucket/*"
      ]
    }
  ]
}
```

Bucket settings for Block Public Access

- Block public access to buckets and objects granted through
 - New access control lists (ACLs)
 - Any access control lists (ACLs)
 - New public bucket or access point policies
- Block public and cross-account access to buckets and objects through any public bucket or access point policies
- These setting were created to prevent company data leaks
- If you know your bucket should never be public, leave these on
- Can be set at the

S3 Security - Other

- Networking:
 - Supports VPC Endpoint (for instances in VPC without www internet)
- Logging and Audit:
 - S3 Access Logs can be stored in other S3 bucket
 - API calls can be looged in AWS CloudTrail
- User Security:
 - MFA Delete: MFA (multi factor authentication) can be required in versioned bucketes to delete object
 - Pre-Sign URLs: URLs that are valid only for a limited time (ex: premium video service for logged in users)

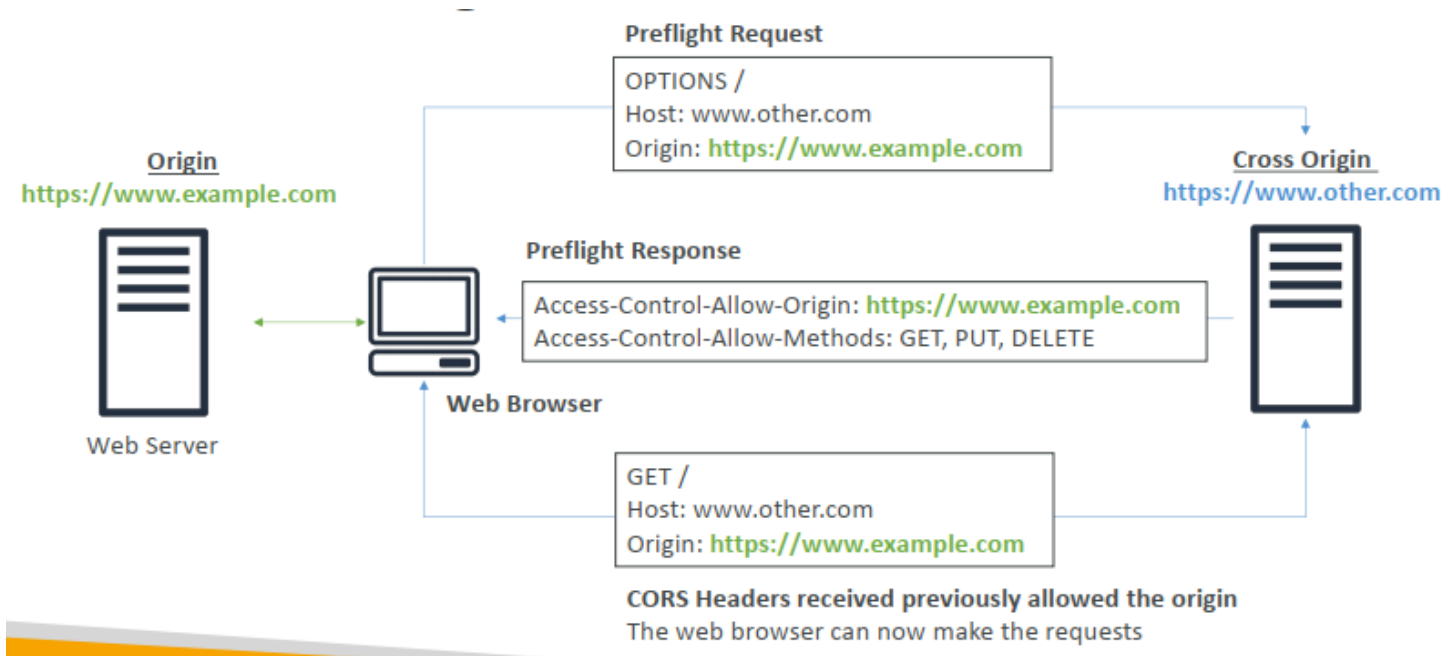
S3 Websites

- S3 can host static websites and have them accessible on the WWW
- The website URL will be:
 - <bucket-name>.s3-website-<AWS-region>.amazonaws.com
 - <bucket-name>.s3-website.<AWS-region>.amazonaws.com
- If you get 403 (Forbidden) error, make sure the bucket policy allow public reads

CORS - Explained

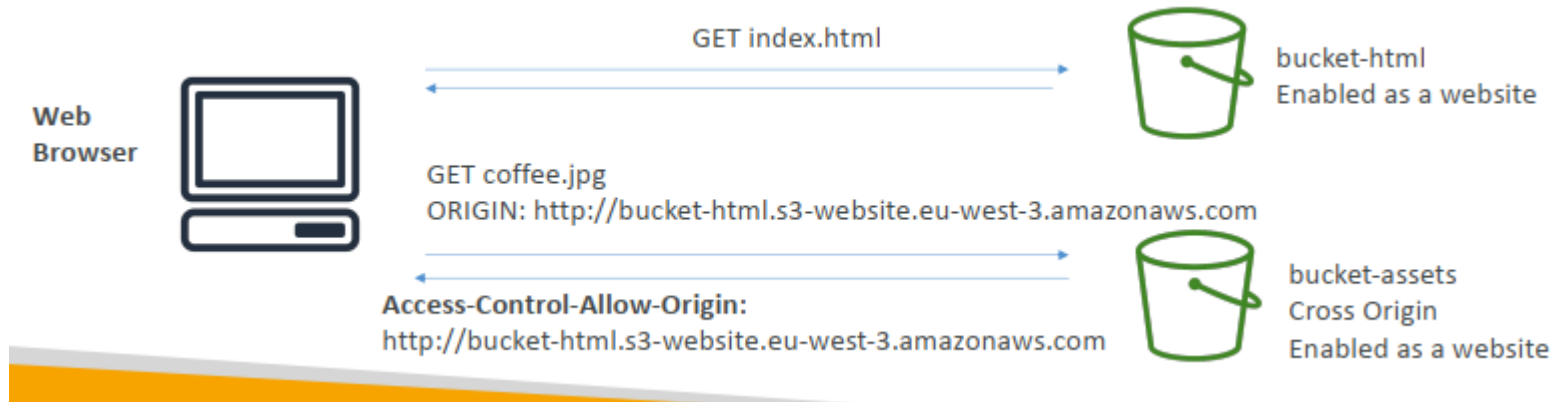
- An origin is scheme (protocol), host (domain) and port
 - Eg: <https://www.example.com> (implied port is 443 for HTTPS, 80 for HTTP)
- CORS means Cross-Origin Resource Sharing
- Web Browser based mechanism to allow requests to other origins while visiting the main origin
- Same origin: <http://example.com/app1> & <http://example.com/app2>
- Different origins: <http://www.example.com> & <http://other.example.com>
- The request won't be fulfilled unless the other origin allows for the request, using **CORS Headers (ex: Access-Control-Allow-Origin)**

CORS - Diagram



S3 CORS

- If a client does a cross-origin request on your S3 bucket, we need to enable the correct CORS headers
- It's a popular exam question
- You can allow for a specific origin or for * (all origins)



Thank you!!!